Repository RAG Lab

A uv-first publication sketch for repository-grounded retrieval, MCP discovery, and deployment handoff

Piece-by-piece implementation map, verification story, and research positioning

A Publication-Style Walkthrough of Repository RAG Lab

OpenAI Codex

March 18, 2026

Abstract

Repository RAG Lab is a compact research scaffold for repository-grounded Retrieval-Augmented Generation (RAG) built around a single implementation shared by notebooks, a Python command-line interface, `make` targets, quality gates, and a thin Rust wrapper. This article describes the project piece by piece, from corpus loading and lexical retrieval to MCP candidate discovery, Azure deployment handoff, notebook playbooks, and verification workflows. The result is not a full production RAG platform; it is a deliberately inspectable lab environment that keeps research, automation, and packaging aligned while leaving clear seams for stronger retrieval and true DSPy compilation later [3, 2, 1].

1 Project Scope

The repository solves a narrow but useful problem: answering questions about a codebase by using the repository itself as the retrieval corpus. The current scope has three parts.

- Repository-grounded question answering over files such as Markdown, Python, Rust, YAML, JSON, and now C headers and sources.
- MCP candidate discovery from repository metadata and package-adjacent hints.
- Azure deployment-manifest generation for downstream workflows that operate outside the repository itself [4].

This means the project is best understood as a cohesive lab scaffold rather than a finished service. It is opinionated about reproducibility and visibility: the same behavior should be reachable through notebooks, tests, the Python CLI, and `make`.

2 Piece-By-Piece Architecture

Layer	Primary files	Responsibility
Corpus loading	`src/repo_rag_lab/corpus.py`	Walk the repository, keep text-like files, skip noisy paths, and load documents into a shared representation.
Retrieval	`src/repo_rag_lab/retrieval.py`	Chunk documents and rank them with a simple lexical overlap baseline.

Answer synthesis	`src/repo_rag_lab/workflow.py`	Assemble top-ranked context into a readable repository-grounded answer.
MCP discovery	`src/repo_rag_lab/mcp.py`	Infer likely MCP-related surfaces from repository and package metadata.
CLI surface	`src/repo_rag_lab/cli.py`	Expose ask, smoke test, MCP discovery, surface verification, and manifest generation through a packaged command.
Notebook scaffolding	`src/repo_rag_lab/notebook_scaffolding.py` and `notebooks/`	Keep demonstrations notebooks tied to reusable package helpers instead of ad hoc notebook-only logic.
Verification	`src/repo_rag_lab/verification.py`, `Makefile`, and `.github/workflows/`	Enforce repository surface expectations and route checks through stable automation layers.
Rust wrapper	`rust-cli/src/main.rs`	Delegate to the Python CLI so the repo can be exercised from a Rust entrypoint without duplicating logic.

3 From Files To Answers

The baseline answer path mirrors the standard RAG structure introduced by Lewis et al. [3], but it keeps the mechanics intentionally small and inspectable.

3.1 Corpus Construction

The repository loader walks the local tree, ignores generated directories such as `artifacts/`, `dist/`, and cache folders, then reads supported text-like files into `RepoDocument` records. The recent hushwheel fixture work widened this surface to include C source and header files, which matters for source-heavy corpora.

3.2 Chunking And Ranking

Documents are chunked into fixed-size text slices and ranked with lexical overlap plus light density weighting. This is explicitly a baseline, not an embeddings-backed system. That choice is important for the project: the current behavior is transparent enough to inspect, benchmark, and replace later.

3.3 Answer Synthesis

The baseline workflow packages the top chunks, the user question, and any MCP hints into a final answer object. The same implementation is reused by smoke tests, the notebooks, and the Rust shim. That consistency is one of the repository's strongest design decisions.

4 DSPy And Research Workflow

The project already depends on DSPy and exposes a DSPy-shaped runtime path, but it does not yet compile optimized DSPy programs or persist tuned artifacts. The current state is closer to a

research preparation environment than a full declarative DSPy pipeline [2].

The supporting pieces are already present:

- training examples in `samples/training/`
- population-planning seeds in `samples/population/`
- notebook helpers for benchmark summaries and logging
- a dedicated DSPy guide in `README.DSPY.MD`

That is enough to explain the future direction clearly. What is missing is the automated teleprompter or optimizer stage and the artifact-loading path that a publication-ready DSPy workflow would normally include.

5 Operational Surfaces

Repository RAG Lab behaves like one project with several front doors:

- `make ask QUESTION="..."` for baseline repository questions
- `make discover-mcp` for repository-local MCP inspection
- `make smoke-test` for an end-to-end sanity pass
- `make verify-surfaces` for notebook and Makefile contract checks
- `make quality` for compile, lint, type, complexity, test, and coverage gates
- `cargo run -manifest-path rust-cli/Cargo.toml - ask -question "..."` for the Rust entrypoint

This publication folder adds one more operational surface: a publication-style article that can be built locally and in GitHub Actions. That matters because the repository already treats docs, checks, and runnable code as one system.

6 Verification Story

The repository's verification loop is unusually explicit for a small research scaffold. Audit notes under `docs/audit/` record what was actually run, `samples/logs/` preserves post-push CI evidence, and the Makefile routes core checks through stable commands.

The current completeness threshold is therefore concrete rather than rhetorical:

- local compile, smoke, and targeted utility tests must pass
- the Python quality gate must clear its coverage threshold
- the Rust wrapper must still build
- GitHub Actions evidence must be captured after pushes

That verification discipline is a meaningful part of the project contribution. It lets a reader trust the scaffold as an executable research artifact rather than a loose notebook dump.

7 Limitations And Next Steps

The project still has visible limits, and the repository now documents them through one shared backlog source. The Markdown backlog and the publication table below are generated from the same `todo-backlog.yaml` file so the working roadmap is visible in both the repository and the article.

Badge	Area	TODO	Linked surfaces	Done when

★	DSPy Training And Runtime	Build the missing automated DSPy training path.	`README.DSPY.MD`
			`src/repo_rag_lab/cli.`
			`Makefile`
				A dedicated training module exists under src/repo_rag_lab/.
				A real dspy.Module plus optimizer flow replaces the current runtime wrapper.
				Compiled programs are versioned under artifacts/dspy/.
				Training is exposed through both the CLI and Makefile.

✎	DSPy Training And Runtime	Make the existing DSPy CLI path operational without custom bootstrap code.	`src/repo_rag_lab/cli.`
			`README.DSPY.MD`
			`tests/test_cli_and_ds`
				LM selection and configuration are handled through flags or documented env setup.
				The –use-dspy path works without ad hoc in-process bootstrap code.
				Tests cover both configured and unconfigured DSPy runs.

✍	DSPy Training And Runtime	Extend the notebooks beyond summary-only DSPy preparation.	`notebooks/03_dspy_tra`
			`src/repo_rag_lab/note`
			`README.DSPY.MD`
				The DSPy training notebook loads and exercises a compiled program.
				Notebook helpers stay aligned with any future compiled-artifact workflow.

Continued on the next page.

Badge	Area	TODO	Linked surfaces	Done when

●	Retrieval, Corpus, And Evaluation	Strengthen retrieval quality beyond the lexical baseline.	`src/repo_rag_lab/retr`
			`src/repo_rag_lab/work`
			`src/repo_rag_lab/dspy`
				A stronger retrieval layer replaces simple lexical overlap.
				Direct file retrieval and MCP-powered retrieval are compared explicitly.
				Retrieval upgrades are shared by both the baseline and DSPy paths.

▲	Retrieval, Corpus, And Evaluation	Expand benchmark and training evidence.	`samples/training/repo`
			`src/repo_rag_lab/benc`
			`tests/test_benchmarks`
				More training examples carry expected_sources evidence.
				Repository-specific benchmark questions and expected answers are expanded.
				Regression and artifact-persistence tests land once training exists.

☞	Retrieval, Corpus, And Evaluation	Keep the benchmark corpus definition stable as the repository grows.	`src/repo_rag_lab/benc`
			`docs/audit/README.md`
			`samples/logs/`
				Operational evidence surfaces stay excluded from the benchmark corpus.
				Benchmark exclusions are reviewed whenever new generated docs or artifacts appear.
				Assertions keep measuring repository content instead of verification byproducts.

✌	Retrieval, Corpus, And Evaluation	Continue improving corpus planning.	`src/repo_rag_lab/popu`
			`samples/population/re`
			`notebooks/04_sample_p`
				Benchmark hit data drives reranking more systematically.
				Population-candidate discovery expands when new repository surfaces matter.

Continued on the next page.

Badge	Area	TODO	Linked surfaces	Done when

✝	Notebook And Documentation Surfaces	Keep notebooks executable as research playbooks.	`notebooks/01_repo_rag`
			`notebooks/03_dspy_tra`
			`src/repo_rag_lab/note`
				Notebook execution checks land in CI if notebook execution becomes an acceptance surface.
				Notebook cell IDs and repository-surface validation stay intact as scaffolding evolves.
				Notebook logging and tuning metadata outputs stay documented under artifacts/.

✖	Notebook And Documentation Surfaces	Keep the DSPy and utility documentation current.	`README.DSPY.MD`
			`utilities/README.md`
			`README.md`
				README.DSPY.MD is refreshed whenever training or benchmark workflows change.
				README.md, utilities/README.md, and notebook guidance match the actual CLI and Make targets.

❖	Verification, CI, And Deployment	Expand verification beyond the current smoke and BDD emphasis.	`tests/test_repository`
			`documentation/azure-d`
			`tests/fixtures/hushwh`
				A live or mocked Azure inference integration test exists.
				Integration-oriented tests are split out or labeled clearly as the repository grows.
				Future DSPy training and artifact-loading paths have dedicated tests.

✧	Verification, CI, And Deployment	Keep CI current with platform changes.	`.github/workflows/ci.`
			`samples/logs/`
			`docs/audit/README.md`
				GitHub Actions dependencies stop emitting Node.js 20 deprecation warnings.
				Post-push CI evidence stays current under samples/logs/.

❑	Verification, CI, And Deployment	Keep packaging and publish surfaces healthy.	`.github/workflows/pub`
			`pyproject.toml`
			`publication/README.md`
				The Publish workflow is re-checked once versioned DSPy artifacts or release tags exist.

Continued on the next page.

Badge	Area	TODO	Linked surfaces	Done when
✯	Repo Hygiene And Workflow Policy	Keep the committed rust-cli/Cargo.lock policy intentional.	`rust-cli/Cargo.lock` `REPO_COMPLETENESS_CHE` `rust-cli/Cargo.toml`	The lockfile is refreshed only when Rust dependencies change. Lockfile drift is treated as repository-surface work rather than incidental noise.
✂	Repo Hygiene And Workflow Policy	Keep audit notes and backlog documents synchronized.	`docs/audit/README.md` `TODO.MD` `samples/logs/`	Audit notes are updated whenever verification status changes materially. The backlog is refreshed when roadmap items are completed, replaced, or reprioritized.

Those gaps do not weaken the core idea. Instead, they define the next layer of work: replace the retriever, formalize DSPy optimization, and keep the same shared workflow contract as the system becomes stronger.

8 Conclusion

Repository RAG Lab is most useful as a transparent, end-to-end scaffold for repository-grounded RAG research. Its value is not a single novel algorithm; it is the coherence between corpus loading, retrieval, workflow entrypoints, notebook playbooks, verification, and packaging. That coherence makes the repository easy to inspect, easy to extend, and easy to explain.

References

[1] Anthropic. Model context protocol. https://modelcontextprotocol.io/, 2024.

[2] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sarthak Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2024.

[3] Patrick Lewis, Ethan Perez, Aleksandras Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[4] Microsoft. Azure AI inference documentation. https://learn.microsoft.com/azure/ai-services/, 2025.